QGENTIC AI

An Automated ML Competition Stack

Reducing Gold-Medal Effort from 200+ Hours to 20

The Problem

200+

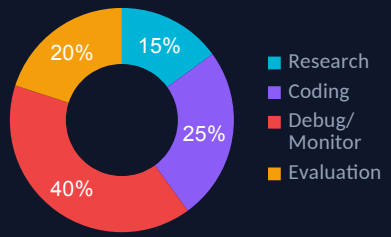
hours for a gold medal

against thousands of competitors

~23 min

context-switch recovery time

- Most time goes to repetitive maintenance: debugging, monitoring, iterating models
- Checking intermediate training results, restarting crashed runs, evaluating submissions
- The bottleneck is human endurance and availability, not intelligence
- Gold-medal work is ~80% disciplined execution, ~20% novel thinking



The Key Insight

200+ hours manual effort

▼

Automated Agent Execution

▼

~20 hrs

90% reduction

~20% **Strategic Thinking**
Novel approaches, competition-specific insights

~80% **Execution Work**
Code gen, debugging, monitoring, evaluation

NOW **LLM Capability**
LLMs can handle structured execution loops

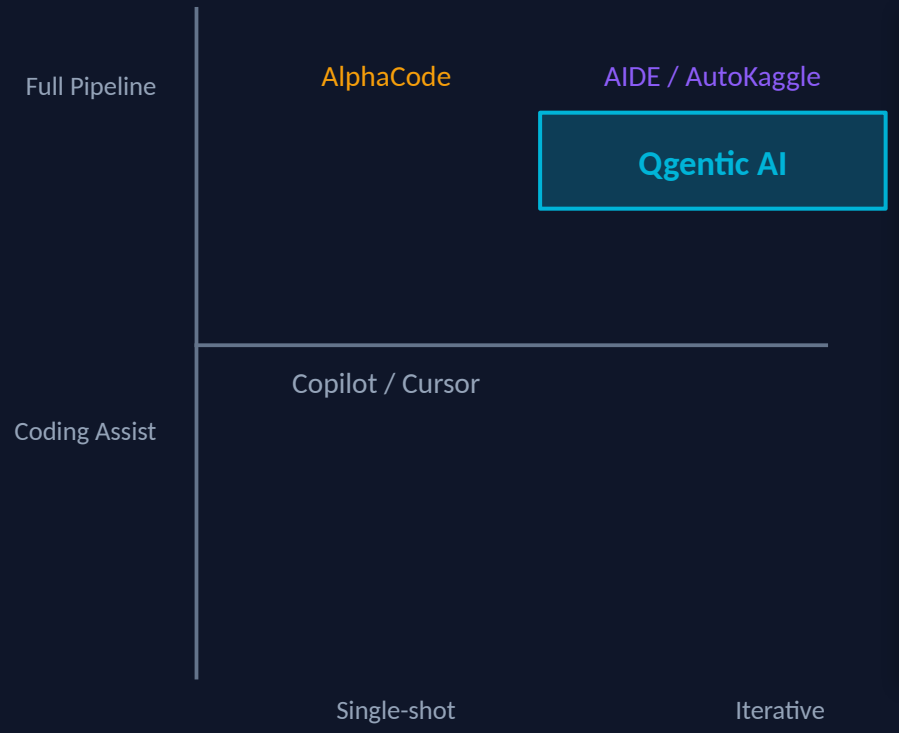
Goal: Human as strategic director, agents handle the grind

AutoML Systems

System	Code Gen	Reasoning	Competition Aware	Multi-Model
Auto-sklearn	✗	✗	✗	✓
AutoGluon	✗	✗	✗	✓
H2O AutoML	✗	✗	✗	✓
Qgentic AI	✓	✓	✓	✓

- AutoML systems optimize within predefined pipelines and fixed search spaces
- No ability to read a competition description and devise a novel approach
- No code generation -- only hyperparameter tuning and model selection
- Qgentic AI combines reasoning, code generation, and competition awareness

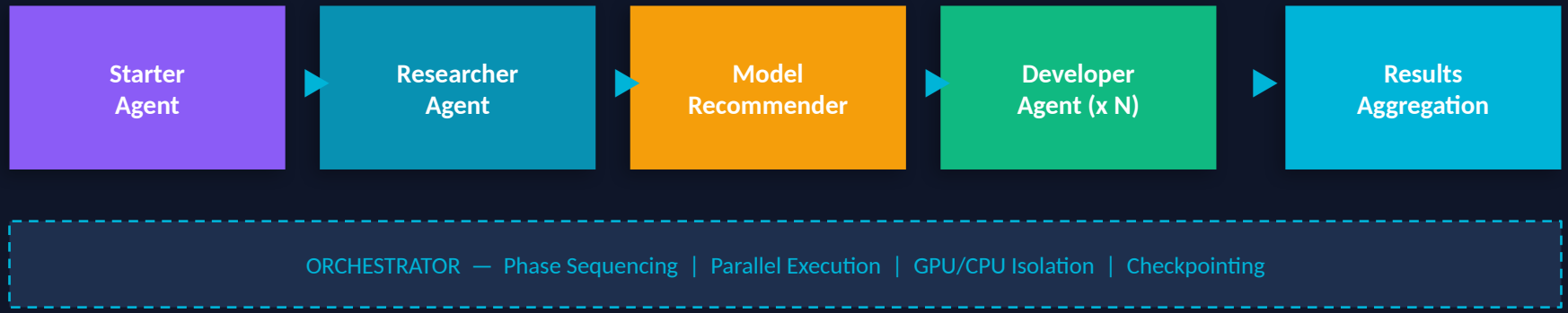
LLM Code Agents



The Gap

- No system combines multi-agent research + model recommendation + parallel execution + cross-model learning
- Copilot/Cursor: code completion, not end-to-end pipeline execution
- AlphaCode/CodeGen: single-shot generation, no iterative refinement
- AIDE, MLE-bench, AutoKaggle: closest, but lack multi-agent coordination and shared learning

System Architecture Overview



LLM Backend

Gemini 3.1 Pro Preview
Structured Pydantic outputs

Tool Calling

Agentic loops with
execute, search, scrape tools

Parallel Workers

Up to N models in parallel
with GPU/CPU isolation

Cross-Model Learning

Shared suggestion pool
across all baselines

StarterAgent — Task Classification

description.md
Competition brief

▶

StarterAgent

Single LLM call
+ Google Search

▶

starter_suggestions.json

```
{
  task_types: ["tabular"],
  summary: "Predict biomass..."
}
```

Tabular

Structured data, feature engineering

NLP

Text classification, NER, summarization

Computer Vision

Image classification, segmentation

Time Series

Forecasting, anomaly detection

Downstream agents use task_types to tailor research, model selection, and code generation

ResearcherAgent — Deep Research Loop

AGENTIC LOOP (up to 512 steps)

`execute_python`

Run EDA code, generate plots
Auto-ingests images (max 6/step)

`read_research_paper`

ArXiv PDF summarization
Extracts methods, results, ablations

`scrape_web_page`

Firecrawl web scraping

Output: plan.md — research plan for downstream agents

ARCHITECTURE

ModelRecommenderAgent — Strategy Selection

Stage 1: LLM selects ~16 candidates (web search enabled)

▼

Stage 2: Paper analysis for all candidates

▼

Stage 3: Refine to ~8 models

Per-Model Recommendations

Preprocessing

Data transforms, augmentation, normalization

Loss Functions

Exactly 1 MUST_HAVE, multiple NICE_TO_HAVE

Hyperparameters

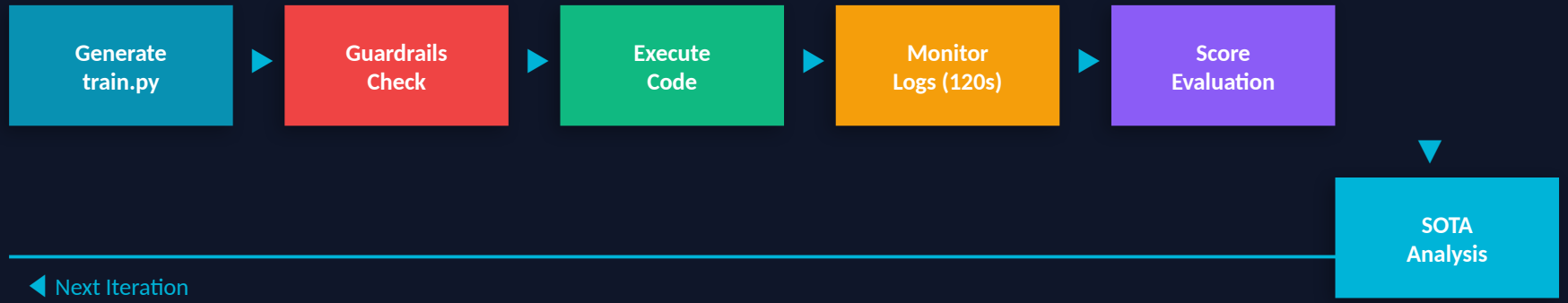
Learning rate, batch size, architecture configs

Inference

MUST_HAVE	= applied in first iteration (NOW)
NICE_TO_HAVE	= fed to SOTA engine for later iterations

METHODOLOGY

DeveloperAgent — Code Generation & Iteration



Log Monitoring

LLM polls every 120s
Detects crashes, NaN losses, stalled training, overfitting

Stack Trace Fix

Fine-tuned model first
Falls back to web search
+ general LLM diagnosis

investigate_library

Reads installed package source code to resolve API questions accurately

Code Injection

Auto-injects CPU affinity, GPU assignment, BASE_DIR, environment setup

Cross-Model Learning

Model A
(e.g., XGBoost)

"Cosine annealing improved score by 0.02"

↔

Model B
(e.g., LightGBM)

"Feature selection with MI improved by 0.015"

↔

Shared Suggestion Pool

`Thread-safe class-level _shared_suggestions`

✓ Cosine annealing (+0.02)

✓ MI feature selection (+0.015)

✗ Label smoothing (-0.01)

✗ Heavy augmentation (-0.005)

Benefits

- Prevents redundant exploration of failed strategies
- Successful techniques propagate to sibling models
- Blacklisted ideas automatically excluded
- Emergent collective optimization across all baselines

Safety & Guardrails

Layer 1: AST Static Analysis

Checks logging.basicConfig order
Detects train/test data partition patterns
Python AST-based, zero LLM cost

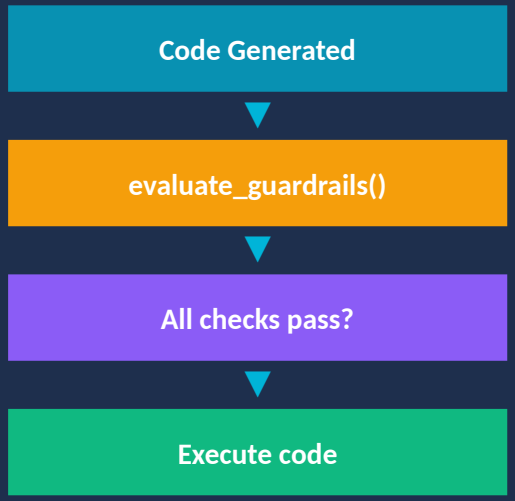
Layer 2: LLM Leakage Review

Gemini 3.1 Pro reviews code for data leakage
Detects train/test contamination
Structured LeakageReviewResponse output

Layer 3: LLM Code Safety

Checks for eval/exec, command injection
Credential leakage detection
Gemini 2.5 Flash (fast, low cost)

Guardrail Pipeline



✗Blocked → feedback to LLM for regeneration

Infrastructure — Parallel Execution

GPU Isolation Modes

MIG

NVIDIA Multi-Instance GPU
Partition single GPU into
independent instances

Multi-GPU

Assign different GPUs to different workers (CUDA_VISIBLE_DEVICES)

CPU-Only

No GPU isolation
CPU affinity pinning via psutil

Resource Pool Architecture

GPU 0	GPU 1	GPU 2	GPU 3
Worker 1	Worker 2	Worker 3	Worker 4

ThreadPoolExecutor + Queue-based dynamic allocation

5 days
Baseline development time budget

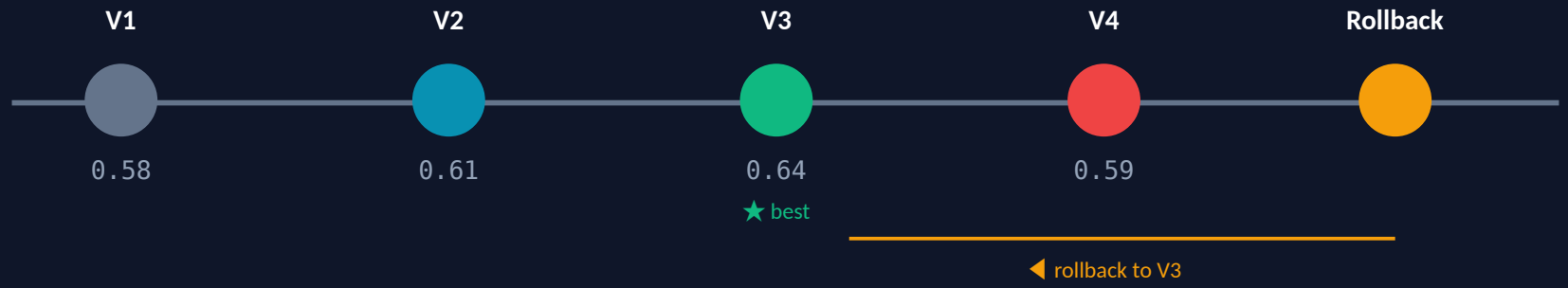
12 hrs
Per-script execution timeout

120s
Log monitoring poll interval

Isolated
Conda environment per model

Checkpoint & Human-in-the-Loop

Version Timeline with Rollback



SQLite Checkpoint System

- Saves full agent state per version
- Scores, suggestions, conversation history
- Rollback: move to _trash/, delete DB rows
- --rollback-to-version N CLI flag

Human-in-the-Loop (INSTRUCTIONS.md)

- # Researcher Instructions — guide research
- # Developer Instructions — guide code gen
- # Models — override model selection
- hitl_sota: true → pause for human approval

CSIRO Biomass Competition

0.63772
Leaderboard Score

32 / 3,802
Final Ranking

Top 1%
Silver Medal

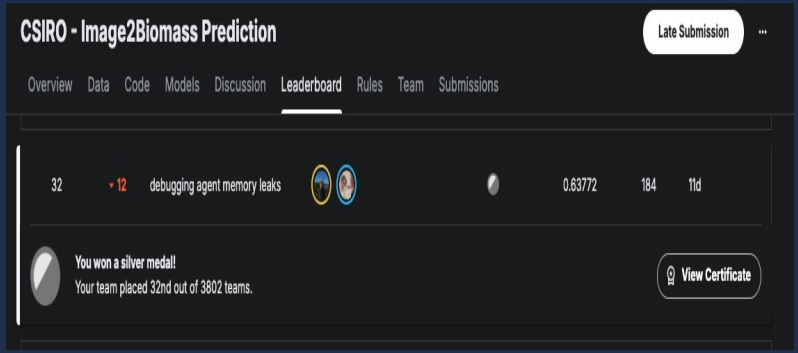
Human Effort Reduction

Traditional: 200+ hours

Qgentic AI: ~20 hrs

90% REDUCTION

Kaggle Leaderboard



Lessons Learned

LLM Log Monitoring Works

Catches overfitting, NaN losses, and stalled training that humans miss during off-hours. Surprisingly effective as a real-time diagnostic tool.

Cross-Model Learning Prevents Redundancy

Without shared state, agents try the same bad idea across all models. The suggestion pool eliminates the 'try the same thing 8 times' failure mode.

Structured Outputs Are Essential

Pydantic schemas for LLM responses make multi-agent communication reliable. Without them, parsing failures cascade through the pipeline.

Robust Retry Logic Is Critical

LLM providers hit 503 during peak demand. Exponential backoff (1s, 5s, 10s, 30s, 60s) + 5-min polling on sustained 503 keeps the system running.

Roadmap & Future Work

Phase 1 — **Observability**

Migrate from Weave to Langfuse for better tracing, cost tracking, and debugging capabilities

Phase 2 — **Enhanced Guardrails**

Pre-execution lint integration, improved leakage detection, richer stack trace remediation

Phase 3 — **Cross-Worker Learning V2**

Richer shared state: embeddings, feature importances, training curves -- not just text suggestions

Phase 4 — **Legal Retrieval**

Competition rules compliance checking via legal retrieval integration (already scaffolded)

★ **Ultimate Goal: Kaggle Competitions Grandmaster**

2 more gold medals needed — let the agents compete while you strategize

Key Takeaways

Multi-agent decomposition beats monolithic agents

Specialized agents with distinct tools > one agent trying to do everything

Cross-model learning creates emergent optimization

Shared suggestion pools enable collective intelligence across parallel baselines

90% effort reduction is achievable today

Current LLM capabilities are sufficient for structured ML execution loops

The future of ML competitions is human + AI collaboration

Humans provide strategic direction; agents handle the disciplined execution

From 200+ hours to 20. Let the agents do the grind.

Open Source: github.com/bogoconic1/Qgentic-AI